

Week 2 - Day 7a: Recursion

Recursion is a method of solving problems where the overall solution depends on *solutions to smaller instances* of the same problem (subproblems). In the context of programming, this would involve *a function calling itself* with different arguments that describe the subproblem. See the following pseudo-code example of a recursive function

```
def recursive_function(parameters):  
    if base_case_condition(parameters):  
        return base_case_value  
    return recursive_function(modified_parameters)
```

Remember, although recursion can be an effective approach for many problems, it's not always the best choice. Recursive solutions are usually short to write, though often less efficient than iterative solutions (i.e. those that use `for` or `while` loops).

Functions in Python have a recursion limit, usually set to 1000. If you exceed this limit, Python will raise a `RecursionError`. You can check your recursion limit with `sys.getrecursionlimit()`, and change it with `sys.setrecursionlimit()`, but it's generally not a good idea to change the limit.

Warm-Up []

Look at this recursive function `multi_print()`. Notice how there are no loops!

```
def multi_print(text, n):  
    if n == 0:  
        return  
    print(text)  
    multi_print(text, n - 1)  
  
multi_print('Hello AddisCoder', 3)
```

What is the recursive case for `multi_print()`?

What is the base case for `multi_print()`? Please use English.

Exercise 1: Fibonacci Number []

Fibonacci numbers have many applications in nature and math. The first two Fibonacci numbers are 0 and 1, and each subsequent number is the sum of the previous two. Hence, the first few Fibonacci numbers are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...

Mathematically, we can write the n -th Fibonacci number, $F(n)$, using a recurrence relation:

$$F(0)=0, F(1)=1$$

and for $n \geq 2$ we have

$$F(n) = F(n-1) + F(n-2)$$

In Week 1, we saw how to implement this sequence **iteratively** using while-loops. This time, let's try to implement **fibonacci recursively**.

Task: Write a recursive function `fibonacci(n)` that returns the n -th Fibonacci number.

```
# TODO: Implement a recursive function to find the n-th Fibonacci number
def fibonacci(n):
    # your code here
    pass

# Test cases
assert fibonacci(0) == 0
assert fibonacci(1) == 1
assert fibonacci(2) == 1
assert fibonacci(3) == 2
assert fibonacci(4) == 3
assert fibonacci(5) == 5
assert fibonacci(6) == 8
assert fibonacci(7) == 13

print("All tests passed. ☑")
```

Exercise 2: Summing a List []

Consider a recursive function called `sum_lst(lst)`. It takes a list of numbers, `lst`, as input and returns the sum of the list. For example:

```
sum_lst([1, 1, 1]) == 3
sum_lst([1, 2, 3]) == 6
sum_lst([]) == 0
sum_lst([1, -1]) == 0
```

2.1 What is the recursive case for `sum_lst`?

2.2 What is the base case for `sum_lst`?

2.3 Use your base case and recursive case to write the `sum_all` function.

```
# TODO: Implement `sum_lst`
def sum_lst(lst):
    # Write your code here
    pass

# Here are some basic test cases to test your solution
assert sum_lst([1, 1, 1]) == 3
```

```

assert sum_lst([1, 2, 3]) == 6
assert sum_lst([]) == 0
assert sum_lst([1, -1]) == 0

print(f"Test cases passed! 🎉")

```

Exercise 3: Counting Letters []

Consider a recursive function called `count_letter(s, letter)`. It takes a string `s` and a single letter `letter` and returns how many times `letter` appears in `s`. For example:

```

count_letter('aaabbbaaab', 'b') == 3
count_letter('aaabbbaaab', 'a') == 6
count_letter('c', 'c') == 1
count_letter('abcd', 'e') == 0
count_letter('', 'a') == 0

```

3.1 What is the recursive case for `count_letter`?

3.2 What is the base case for `count_letter`?

3.3 Use your base and recursive case to implement the `count_letter` function

```

# TODO: Implement `count_letter`
def count_letter(s, letter):
    #Write your code here
    pass

# Here are some test cases to test your solution
assert count_letter('aaabbbaaab', 'b') == 3
assert count_letter('aaabbbaaab', 'a') == 6
assert count_letter('c', 'c') == 1
assert count_letter('abcd', 'e') == 0
assert count_letter('', 'a') == 0

print(f"Test cases passed! 🎉")

```

Exercise 4: Exponential Function []

Use recursion to write an exponential function, `exp(a, b)`, that calculates a^b . Some examples:

```

exp(5, 2)    == 25
exp(5, 3)    == 125
exp(2, 3)    == 8
exp(1, 100)  == 1
exp(5, 0)    == 1
exp(123, 0)  == 1

```

Assume a and b are integers, $a \geq 1$, and $b \geq 0$. Note that under these assumptions we know $a^0 = 1$ and $a^b = a \cdot a^{b-1}$.

4.1 What is the recursive case for `exp(a, b)`?

Hint: Think about how you can break down the problem into smaller subproblems. The assumptions and implications above should help you.

4.2 What is the base case for `exp(a, b)`?

Hint: Think about how you can break down the problem into smaller subproblems. The assumptions and implications above should help you.

4.3 Use your base and recursive case to write the `exp(a, b)` function.

```
# TODO: Implement `exp`
def exp(a, b):
    # Write your code here
    pass

# Here are some test cases to test your solution
assert exp(5, 2) == 25
assert exp(5, 3) == 125
assert exp(2, 3) == 8
assert exp(1, 100) == 1
assert exp(5, 0) == 1
assert exp(171, 0) == 1

print(f"Test cases passed! 🎉")
```

Exercise 5: Reverse a String [] []

Implement a recursive function called `reverse(s)` that takes a string, `s`, and reverses it. For example:

```
reverse('abc') == 'cba'
reverse('ccc') == 'ccc'
reverse('this is a long string') == 'gnirts gnol a si siht'
reverse('') == ''
```

5.1 What is the recursive case for `reverse(s)`?

5.2 What is the base case for `reverse(s)`?

5.3 Using your base and recursive case write code for the `reverse(s)` function

```
# TODO: Implement `reverse`
def reverse(s):
    pass
```

```
# Here are some test cases to test your solution
assert reverse('abc') == 'cba'
assert reverse('ccc') == 'ccc'
assert reverse('this is a long string') == 'gnirts gnol a si siht'
assert reverse('') == ''

print(f"Test cases passed! 🎉")
```

Exercise 6: Remove a Letter [] []

Define a recursive function `remove_letter(s, letter)` which takes a string `s` and a single character `letter` and returns the string back without the given letter. For example:

```
remove_letter('abc', 'a') == 'bc'
remove_letter('abc', 'o') == 'abc'
remove_letter('', 'a') == ''
```

6.1 What is the recursive case for `remove_letter(s)`?

6.2 What is the base case for `remove_letter(s)`?

6.3 Using your base and recursive case write code for the `remove_letter(s)` function

```
# TODO: Implement `remove_letter`
def remove_letter(s, letter):
    # Write your code here
    pass

# Here are some test cases to test your solution
assert remove_letter('abc', 'a') == 'bc'
assert remove_letter('abaca', 'a') == 'bc'
assert remove_letter('abc', 'd') == 'abc'
assert remove_letter('', 'a') == ''
assert remove_letter('letter', 't') == 'leer'

print(f"Test cases passed! 🎉")
```

Congratulations 🎉